CCEE Phase 2 Cheatsheet: NoSQL & MongoDB

Sessions 10–15 (Comprehensive)

1. NoSQL Concepts (Sessions 10 & 11)

RDBMS vs. NoSQL Architecture

RDBMS (SQL)	NoSQL (Not Only SQL)
Scaling: Vertical (Scale Up - add RAM/CPU).	**Scaling:** Horizontal (Scale Out - add servers/Sharding).
Schema: Rigid/Fixed (Define before insert).	**Schema:** Dynamic/Schemaless.
Theorem: ACID (Atomicity, Consistency...).	**Theorem:** CAP (Base on BASE properties).
Structure: Tables, Rows, Foreign Keys.	**Structure:** Documents, Key-Value, Graphs.

The CAP Theorem (Pick only 2)

- **C (Consistency):** All nodes see the same data at the same time.
- **A (Availability):** Every request gets a response (even if data is old).
- **P (Partition Tolerance):** System works even if network breaks between nodes.
- **RDBMS:** CA (Fails if network partitions).
- **MongoDB/HBase:** CP (Consistent + Partition Tolerant).
- **Cassandra/Dynamo:** AP (Available + Partition Tolerant).

NoSQL Categories (Session 12 Extra)

- **Key-Value (e.g., Redis):** Fastest. Stores data in a HashMap. Good for caching/sessions.
- **Column-Family (e.g., HBase, Cassandra):** Wide-column store. Good for massive write throughput on Big Data.
- **Document (e.g., MongoDB):** Stores JSON-like documents. Flexible.
- **Graph (e.g., Neo4j):** Stores nodes and edges. Best for social networks.

2. MongoDB Basics (Session 12)

Terminology Translation

- Database → Database
- Table → **Collection**
- Row → **Document** (BSON format - Binary JSON)
- Column → **Field**
- Primary Key → **_id** (Created automatically as ObjectId if not provided)

HBase & Redis (Syllabus Specifics)

Exam Differentiators

Redis: In-Memory Key-Value store.

- **Speed:** Microsecond latency (RAM).
- **Commands:** `SET key value`, `GET key`, `INCR counter`.

HBase: Column-oriented, runs on Hadoop (HDFS).

- **Access:** Random real-time read/write access to Big Data.
- **Schema:** Must define "Column Families" in advance, but columns are dynamic.

3. MongoDB Operations (Sessions 12 & 15)

CRUD Syntax

```
// INSERT
db.users.insertOne({ name: "Rohan", age: 25, city: "Pune" })

// READ (SELECT)
db.users.find() // Select *
db.users.find({ age: 25 }) // WHERE age = 25
db.users.find({ age: { $gt: 20 } }) // WHERE age > 20

// UPDATE
// Syntax: updateOne(filter, update_action)
// TRAP: If you forget $set, it REPLACES the whole document!
db.users.updateOne(
   { name: "Rohan" },
   { $set: { age: 26 } }
)

// DELETE
db.users.deleteOne({ name: "Rohan" })
```

Common Operators

- **Comparison:** `$eq`, `$gt` ($>$), `$gte` ($\geq$), `$lt` ($<$), `$lte` ($\leq$), `$ne` ($\neq$), `$in` (in array).
- **Logical:** `$or`, `$and`, `$not`, `$nor`.
- **Update:** `$set` (update field), `$inc` (increment number), `$push` (add to array), `$unset` (remove field).

4. Internals & Indexing (Sessions 13 & 14)

Why Index?

- **Without Index:** "Collection Scan" (Checks every doc). O(N). Slow.
- **With Index:** "B-Tree Scan" (Jumps to location). O(log N). Fast.
- **Cost:** Indexes consume RAM and disk space. They **slow down writes** (Insert/Update/Delete) because the index must be updated too.

Managing Indexes

```
// Create Index (1 = Ascending, -1 = Descending)
db.students.createIndex({ "score": 1 })

// Compound Index (Order matters!)
db.students.createIndex({ "city": 1, "score": -1 })

// Unique Index (Enforces uniqueness like SQL Unique Constraint)
db.users.createIndex({ "email": 1 }, { unique: true })
```

The ESR Rule (Optimizing Compound Indexes)

For efficient queries, order index keys as:

1. **E**quality: Fields you match exactly ($name = "A"$).
2. **S**ort: Fields you sort by ($sort = \{date : 1\}$).
3. **R**ange: Fields you filter by range ($price > 500$).

5. Aggregation Framework (Session 15)

Concept: A pipeline of stages processing data like a conveyor belt.

```
db.orders.aggregate([
   // Stage 1: Filter (WHERE)
   { $match: { status: "A" } },

   // Stage 2: Group (GROUP BY)
   // _id is MANDATORY in group
   { $group: { _id: "$cust_id", total: { $sum: "$amount" } } },

   // Stage 3: Sort (ORDER BY)
   { $sort: { total: -1 } }
])
```